

Programação Funcional

1º Ano – LCC/LEF/LEI

Questões

1. Apresente uma definição recursiva da função (pré-definida) `enumFromTo :: Int -> Int -> [Int]` que constrói a lista dos números inteiros compreendidos entre dois limites.

Por exemplo, `enumFromTo 1 5` corresponde à lista `[1,2,3,4,5]`

2. Apresente uma definição recursiva da função (pré-definida) `enumFromThenTo :: Int -> Int -> Int -> [Int]` que constrói a lista dos números inteiros compreendidos entre dois limites e espaçados de um valor constante.

Por exemplo, `enumFromThenTo 1 3 10` corresponde à lista `[1,3,5,7,9]`.

3. Apresente uma definição recursiva da função (pré-definida) `(++) :: [a] -> [a] -> [a]` que concatena duas listas.

Por exemplo, `(++) [1,2,3] [10,20,30]` corresponde à lista `[1,2,3,10,20,30]`.

4. Apresente uma definição recursiva da função (pré-definida) `(!!) :: [a] -> Int -> a` que dada uma lista e um inteiro, calcula o elemento da lista que se encontra nessa posição (assume-se que o primeiro elemento se encontra na posição 0).

Por exemplo, `(!!) [10,20,30] 1` corresponde a 20.

Ignore os casos em que a função não se encontra definida (i.e., em que a posição fornecida não corresponde a nenhuma posição válida da lista).

5. Apresente uma definição recursiva da função (pré-definida) `reverse :: [a] -> [a]` que dada uma lista calcula uma lista com os elementos dessa lista pela ordem inversa.

Por exemplo, `reverse [10,20,30]` corresponde a `[30,20,10]`.

6. Apresente uma definição recursiva da função (pré-definida) `take :: Int -> [a] -> [a]` que dado um inteiro `n` e uma lista `l` calcula a lista com os (no máximo) `n` primeiros elementos de `l`.

A lista resultado só terá menos de que `n` elementos se a lista `l` tiver menos do que `n` elementos. Nesse caso a lista calculada é igual à lista fornecida.

Por exemplo, `take 2 [10,20,30]` corresponde a `[10,20]`.

7. Apresente uma definição recursiva da função (pré-definida) `drop :: Int -> [a] -> [a]` que dado um inteiro `n` e uma lista `l` calcula a lista sem os (no máximo) `n` primeiros elementos de `l`.

Se a lista fornecida tiver `n` elementos ou menos, a lista resultante será vazia.

Por exemplo, `drop 2 [10,20,30]` corresponde a `[30]`.

8. Apresente uma definição recursiva da função (pré-definida) `zip :: [a] -> [b] -> [(a,b)]` constói uma lista de pares a partir de duas listas.
Por exemplo, `zip [1,2,3] [10,20,30,40]` corresponde a `[(1,10),(2,20),(3,30)]`.
9. Apresente uma definição recursiva da função (pré-definida) `replicate :: Int -> a -> [a]` que dado um inteiro `n` e um elemento `x` constói uma lista com `n` elementos, todos iguais a `x`.
Por exemplo, `replicate 3 10` corresponde a `[10,10,10]`.
10. Apresente uma definição recursiva da função (pré-definida) `intersperse :: a -> [a] -> [a]` que dado um elemento e uma lista, constrói uma lista em que o elemento fornecido é *intercalado* entre os elementos da lista fornecida.
Por exemplo, `intersperse 1 [10,20,30]` corresponde a `[10,1,20,1,30]`.
11. Apresente uma definição recursiva da função (pré-definida) `group :: Eq a => [a] -> [[a]]` que agrupa elementos iguais e consecutivos de uma lista.
Por exemplo, `group [1,2,2,3,4,4,4,5,4]` corresponde a `[[1],[2,2],[3],[4,4,4],[5],[4]]`.
12. Apresente uma definição recursiva da função (pré-definida) `concat :: [[a]] -> [a]` que concatena as listas de uma lista.
Por exemplo, `concat [[1],[2,2],[3],[4,4,4],[5],[4]]` corresponde a `[1,2,2,3,4,4,4,5,4]`.
13. Apresente uma definição recursiva da função (pré-definida) `inits :: [a] -> [[a]]` que calcula a lista dos prefixos de uma lista.
Por exemplo, `inits [11,21,13]` corresponde a `[[],[11],[11,21],[11,21,13]]`.
14. Apresente uma definição recursiva da função (pré-definida) `tails :: [a] -> [[a]]` que calcula a lista dos sufixos de uma lista.
Por exemplo, `tails [1,2,3]` corresponde a `[[1,2,3],[2,3],[3],[]]`.
15. Defina a função `heads :: [[a]] -> [a]` que recebe uma lista de listas e produz a lista com o primeiro elemento de cada lista.
Por exemplo, `heads [[2,3,4],[1,7],[],[8,5,3]]` corresponde a `[2,1,8]`.
16. Defina a função `total :: [[a]] -> Int` que recebe uma lista de listas e conta o total de elementos (de todas as listas)
Por exemplo, `total [[2,3,4],[1,7],[],[8,5,3]]` corresponde a 8.
17. Defina a função `fun :: (a,b,c) -> [(a,c)]` que recebe uma lista de triplos e produz a lista de pares com o primeiro e o terceiro elemento de cada triplo.
Por exemplo, `fun [("rui",3,2), ("maria",5,2), ("ana",43,7)]` corresponde a `[("rui",2), ("maria",2), ("ana",7)]`.
18. Defina a função `cola :: [(String,b,c)] -> String` que recebe uma lista de triplos e concatena as strings que estão na primeira componente dos triplos.
Por exemplo, `cola [("rui",3,2), ("maria",5,2), ("ana",43,7)]` corresponde a "ruimariaana".

19. Defina a função `idade :: Int -> Int -> [(String,Int)] -> [String]` que recebe o ano, a idade e uma lista de pares com o nome e o ano de nascimento de cada pessoa, e devolve a listas de nomes das pessoas que nesse ano atingirão ou já ultrapassaram a idade indicada.

Por exemplo, `idade 2021 26 [("rui",1995), ("maria",2009), ("ana",1947)]` corresponde a `["rui","ana"]`.

20. Apresente uma definição recursiva da função,

`powerEnumFrom :: Int -> Int -> [Int]`

que dado um valor n e um valor m constrói a lista $[n^0, \dots, n^{m-1}]$.

21. Apresente uma definição recursiva da função,

`isPrime :: Int -> Bool`

que dado um número inteiro maior ou igual a 2 determina se esse número é primo. Para determinar se um número n é primo, descubra se existe algum número inteiro m tal que $2 \leq m \leq \sqrt{n}$ e $\text{mod } n \ m = 0$. Se um tal número não existir então n é primo, e se existir então n não é primo.

22. Apresente uma definição recursiva da função (pré-definida) `isPrefixOf :: Eq a => [a] -> [a] -> Bool` que testa se uma lista é prefixo de outra.

Por exemplo, `isPrefixOf [10,20] [10,20,30]` corresponde a `True` enquanto que `isPrefixOf [10,30] [10,20,30]` corresponde a `False`.

23. Apresente uma definição recursiva da função (pré-definida) `isSuffixOf :: Eq a => [a] -> [a] -> Bool` que testa se uma lista é sufixo de outra.

Por exemplo, `isSuffixOf [20,30] [10,20,30]` corresponde a `True` enquanto que `isSuffixOf [10,30] [10,20,30]` corresponde a `False`.

24. Apresente uma definição recursiva da função (pré-definida) `isSubsequenceOf :: Eq a => [a] -> [a] -> Bool` que testa se os elementos de uma lista ocorrem noutra pela mesma ordem relativa.

Por exemplo, `isSubsequenceOf [20,40] [10,20,30,40]` corresponde a `True` enquanto que `isSubsequenceOf [40,20] [10,20,30,40]` corresponde a `False`.

25. Apresente uma definição recursiva da função (pré-definida) `elemIndices :: Eq a => a -> [a] -> [Int]` que calcula a lista de posições em que um dado elemento ocorre numa lista.

Por exemplo, `elemIndices 3 [1,2,3,4,3,2,3,4,5]` corresponde a `[2,4,6]`.

26. Apresente uma definição recursiva da função (pré-definida) `nub :: Eq a => [a] -> [a]` que calcula uma lista com os mesmos elementos da recebida, sem repetições.

Por exemplo, `nub [1,2,1,2,3,1,2]` corresponde a `[1,2,3]`.

27. Apresente uma definição recursiva da função (pré-definida) `delete :: Eq a => a -> [a] -> [a]` que retorna a lista resultante de remover (a primeira ocorrência de) um dado elemento de uma lista.
- Por exemplo, `delete 2 [1,2,1,2,3,1,2]` corresponde a `[1,1,2,3,1,2]`. Se não existir nenhuma ocorrência a função deverá retornar a lista recebida.
28. Apresente uma definição recursiva da função (pré-definida) `(\\) :: Eq a => [a] -> [a] -> [a]` que retorna a lista resultante de remover (as primeiras ocorrências) dos elementos da segunda lista da primeira.
- Por exemplo, `(\\) [1,2,3,4,5,1] [1,5]` corresponde a `[2,3,4,1]`.
29. Apresente uma definição recursiva da função (pré-definida) `union :: Eq a => [a] -> [a] -> [a]` que retorna a lista resultante de acrescentar à primeira lista os elementos da segunda que não ocorrem na primeira.
- Por exemplo, `union [1,1,2,3,4] [1,5]` corresponde a `[1,1,2,3,4,5]`.
30. Apresente uma definição recursiva da função (pré-definida) `intersect :: Eq a => [a] -> [a] -> [a]` que retorna a lista resultante de remover da primeira lista os elementos que não pertencem à segunda.
- Por exemplo, `intersect [1,1,2,3,4] [1,3,5]` corresponde a `[1,1,3]`.
31. Apresente uma definição recursiva da função (pré-definida) `insert :: Ord a => a -> [a] -> [a]` que dado um elemento e uma lista ordenada retorna a lista resultante de inserir ordenadamente esse elemento na lista.
- Por exemplo, `insert 25 [1,20,30,40]` corresponde a `[1,20,25,30,40]`.
32. Apresente uma definição recursiva da função (pré-definida) `unwords :: [String] -> String` que junta todas as strings da lista numa só, separando-as por um espaço.
- Por exemplo, `unwords ["Programacao", "Funcional"]` corresponde a `"Programacao Funcional"`.
33. Apresente uma definição recursiva da função (pré-definida) `unlines :: [String] -> String` que junta todas as strings da lista numa só, separando-as pelo caracter `'\n'`.
- Por exemplo, `unlines ["Prog", "Func"]` corresponde a `"Prog\nFunc\n"`.
34. Apresente uma definição recursiva da função `pMaior :: Ord a => [a] -> Int` que dada uma lista não vazia, retorna a posição onde se encontra o maior elemento da lista. As posições da lista começam em 0, i.e., a função deverá retornar 0 se o primeiro elemento da lista for o maior.
35. Apresente uma definição recursiva da função (pré-definida) `lookup :: Eq a => a -> [(a,b)] -> Maybe b` que retorna uma lista construída a partir de elementos de uma lista (o segundo argumento) atendendo a uma condição dada pelo primeiro argumento.
- Por exemplo, `lookup 'a' [( 'a',1), ( 'b',4), ( 'c',5)]` corresponde à lista `Just 1`.
36. Defina a função `preCrescente :: Ord a => [a] -> [a]` calcula o maior prefixo crescente de uma lista.
- Por exemplo, `preCrescente [3,7,9,6,10,22]` corresponde a `[3,7,9]`.

37. Apresente uma definição recursiva da função `iSort :: Ord a => [a] -> [a]` que calcula o resultado de ordenar uma lista. Assuma, se precisar, que existe definida a função `insert :: Ord a => a -> [a] -> [a]` que dado um elemento e uma lista ordenada retorna a lista resultante de inserir ordenadamente esse elemento na lista.
38. Apresente uma definição recursiva da função `menor :: String -> String -> Bool` que dadas duas strings, retorna `True` se e só se a primeira for menor do que a segunda, segundo a ordem lexicográfica (i.e., do dicionário)
- Por exemplo, `menor "sai" "saiu"` corresponde a `True` enquanto que `menor "programacao" "funcional"` corresponde a `False`.
39. Considere que se usa o tipo `[(a,Int)]` para representar multi-conjuntos de elementos de `a`. Considere ainda que nestas listas não há pares cuja primeira componente coincida, nem cuja segunda componente seja menor ou igual a zero.
- Defina a função `elemMSet :: Eq a => a -> [(a,Int)] -> Bool` que testa se um elemento pertence a um multi-conjunto.
- Por exemplo, `elemMSet 'a' [( 'b',2), ( 'a',4), ( 'c',1)]` corresponde a `True` enquanto que `elemMSet 'd' [( 'b',2), ( 'a',4), ( 'c',1)]` corresponde a `False`.
40. Considere que se usa o tipo `[(a,Int)]` para representar multi-conjuntos de elementos de `a`. Considere ainda que nestas listas não há pares cuja primeira componente coincida, nem cuja segunda componente seja menor ou igual a zero.
- Defina a função `converteMSet :: [(a,Int)] -> [a]` que converte um multi-conjunto na lista dos seus elementos
- Por exemplo, `converteMSet [( 'b',2), ( 'a',4), ( 'c',1)]` corresponde a `"bbaaaaac"`.
41. Considere que se usa o tipo `[(a,Int)]` para representar multi-conjuntos de elementos de `a`. Considere ainda que nestas listas não há pares cuja primeira componente coincida, nem cuja segunda componente seja menor ou igual a zero.
- Defina a função `insereMSet :: Eq a => a -> [(a,Int)] -> [(a,Int)]` que acrescenta um elemento a um multi-conjunto.
- Por exemplo, `insereMSet 'c' [( 'b',2), ( 'a',4), ( 'c',1)]` corresponde a `[( 'b',2), ( 'a',4), ( 'c',2)]`.
42. Considere que se usa o tipo `[(a,Int)]` para representar multi-conjuntos de elementos de `a`. Considere ainda que nestas listas não há pares cuja primeira componente coincida, nem cuja segunda componente seja menor ou igual a zero.
- Defina a função `removeMSet :: Eq a => a -> [(a,Int)] -> [(a,Int)]` que remove um elemento a um multi-conjunto. Se o elemento não existir, deve ser retornado o multi-conjunto recebido.
- Por exemplo, `removeMSet 'c' [( 'b',2), ( 'a',4), ( 'c',1)]` corresponde a `[( 'b',2), ( 'a',4)]`.
43. Considere que se usa o tipo `[(a,Int)]` para representar multi-conjuntos de elementos de `a`. Considere ainda que nestas listas não há pares cuja primeira componente coincida, nem cuja segunda componente seja menor ou igual a zero.

Defina a função `constroiMSet :: Ord a => [a] -> [(a,Int)]` dada uma lista ordenada por ordem crescente, calcula o multi-conjunto dos seus elementos.

Por exemplo, `constroiMSet "aaabccc"` corresponde a `[('a',3), ('b',1), ('c',3)]`.

44. Apresente uma definição recursiva da função pré-definida `partitionEithers :: [Either a b] -> ([a],[b])` que divide uma lista de *Eithers* em duas listas.

45. Apresente uma definição recursiva da função pré-definida `catMaybes :: [Maybe a] -> [a]` que coleciona os elementos do tipo *a* de uma lista.

46. Considere o seguinte tipo para representar movimentos de um robot.

```
data Movimento = Norte | Sul | Este | Oeste
    deriving Show
```

Defina a função `caminho :: (Int,Int) -> (Int,Int) -> [Movimento]` que, dadas as posições inicial e final (coordenadas) do robot, produz uma lista de movimentos suficientes para que o robot passe de uma posição para a outra.

47. Consider o seguinte tipo de dados,

```
data Movimento = Norte | Sul | Este | Oeste
    deriving Show
```

Defina a função `hasLoops :: (Int,Int) -> [Movimento] -> Bool` que dada uma posição inicial e uma lista de movimentos (correspondentes a um percurso) verifica se o robot alguma vez volta a passar pela posição inicial ao longo do percurso correspondente. Pode usar a função `posicao` definida acima.

48. Considere os seguintes tipos para representar pontos e rectângulos, respectivamente. Assuma que os rectângulos têm os lados paralelos aos eixos e são representados apenas por dois dos pontos mais afastados.

```
type Ponto = (Float,Float)
data Rectangulo = Rect Ponto Ponto
```

Defina a função `contaQuadrados :: [Rectangulo] -> Int` que, dada uma lista com rectângulos, conta quantos deles são quadrados.

49. Considere os seguintes tipos para representar pontos e rectângulos, respectivamente. Assuma que os rectângulos têm os lados paralelos aos eixos e são representados apenas por dois dos pontos mais afastados.

```
type Ponto = (Float,Float)
data Rectangulo = Rect Ponto Ponto
```

Defini a função `areaTotal :: [Rectangulo] -> Float` que, dada uma lista com rectângulos, determina a área total que eles ocupam.

50. Considere o seguinte tipo para representar o estado de um equipamento.

```
data Equipamento = Bom | Razoavel | Avariado
    deriving Show
```

Defina a função `naoReparar :: [Equipamento] -> Int` que determina a quantidade de equipamentos que não estão avariados.